

SYSTÈMES ROBOTIQUES et INTERACTIFS



U Université
de Toulouse

Projet : Les pirates

Diagramme de classe et de séquence Système

Réalisé par :

➤ **BACHAA Hajar**

Année universitaire 2025-2026

Introduction

Ce projet consiste à modéliser un jeu de plateau en utilisant UML.

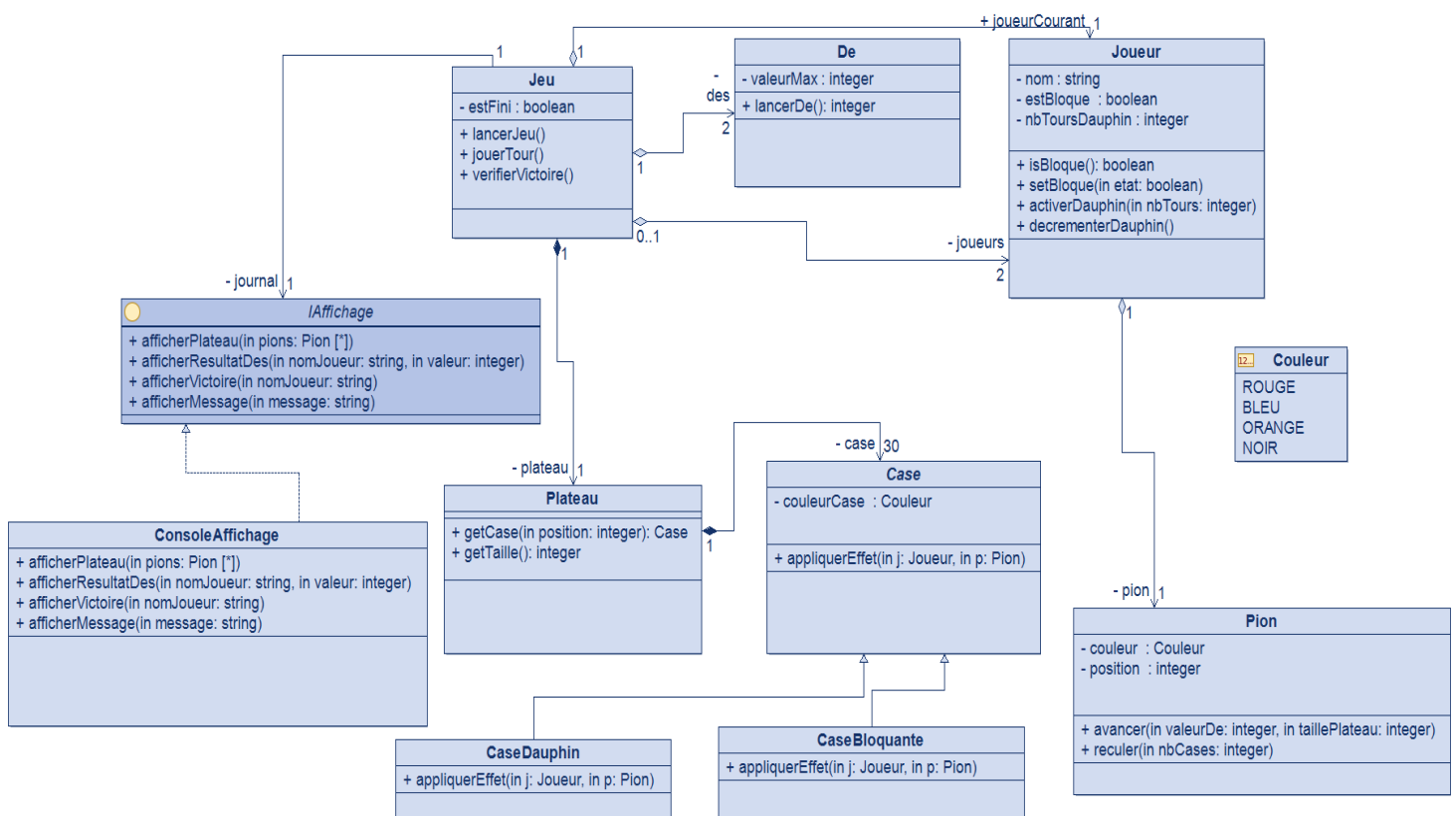
Le jeu comprend des joueurs, des pions, un plateau composé de différentes cases spéciales (case bloquante, case dauphin et case bombe), ainsi que des dés permettant le déplacement.

L'objectif de ce rapport est de présenter :

- Le diagramme de classes,
- Les diagrammes de séquence principaux,

Et d'expliquer les choix de conception.

Diagramme de classe

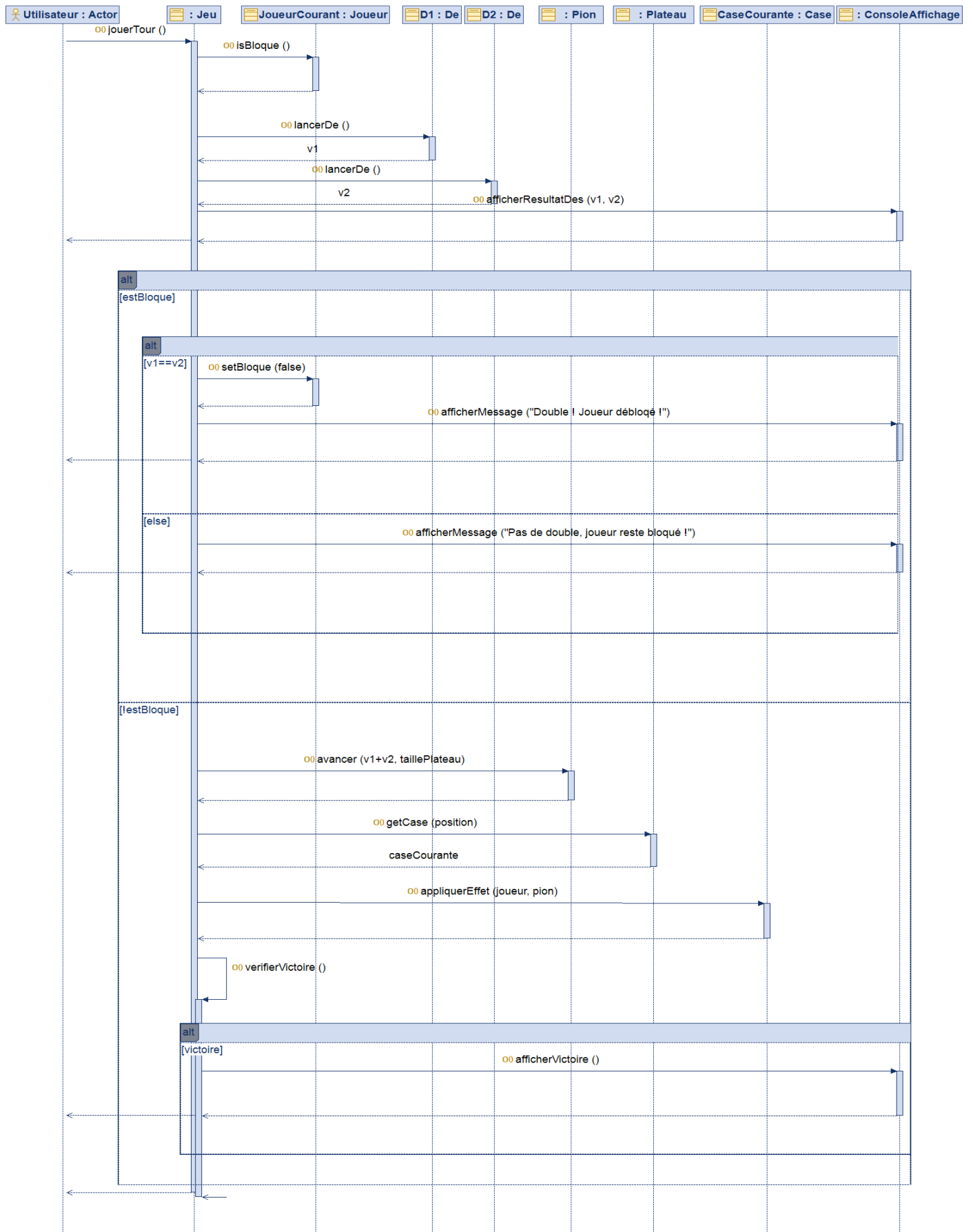


Diagrammes de séquence global

Ce diagramme de séquence représente le déroulement complet d'un tour de jeu.

Le joueur déclenche la méthode `jouerTour()` de la classe `Jeu`.

Le système commence par vérifier si le joueur est bloqué via la méthode `isBloque()`.



▪ **Cas 1 : Joueur bloqué**

Si le joueur est bloqué, les deux dés sont lancés.

Le résultat est affiché, puis une structure conditionnelle (alt) vérifie si les deux valeurs sont identiques.

- Si un double est obtenu ($v1 == v2$), la méthode `setBloque(false)` est appelée et le joueur est débloqué.
- Sinon, un message indique que le joueur reste bloqué et le tour se termine.

▪ **Cas 2 : Joueur non bloqué**

Si le joueur n'est pas bloqué, le pion avance d'un nombre de cases égal à la somme des deux dés.

Le jeu récupère ensuite la case correspondante via `getCase(position)` et appelle `appliquerEffet(joueur, pion)` afin d'exécuter le comportement spécifique de la case.

Enfin, la méthode `verifierVictoire()` est appelée.

Si la condition de victoire est remplie, un message de victoire est affiché.

Diagrammes de séquence - Case spéciale : Bombe

J'ai ajouté la Case Bombe qui fait perdre une vie au joueur. Chaque joueur commence la partie avec 5 cœurs. Si un joueur a le malheur de tomber 5 fois sur une bombe, il perd tous ses cœurs d'un coup. Dans ce cas, le joueur est éliminé et la partie s'arrête immédiatement pour lui. C'est une façon de perdre le jeu avant même d'arriver à la fin du plateau

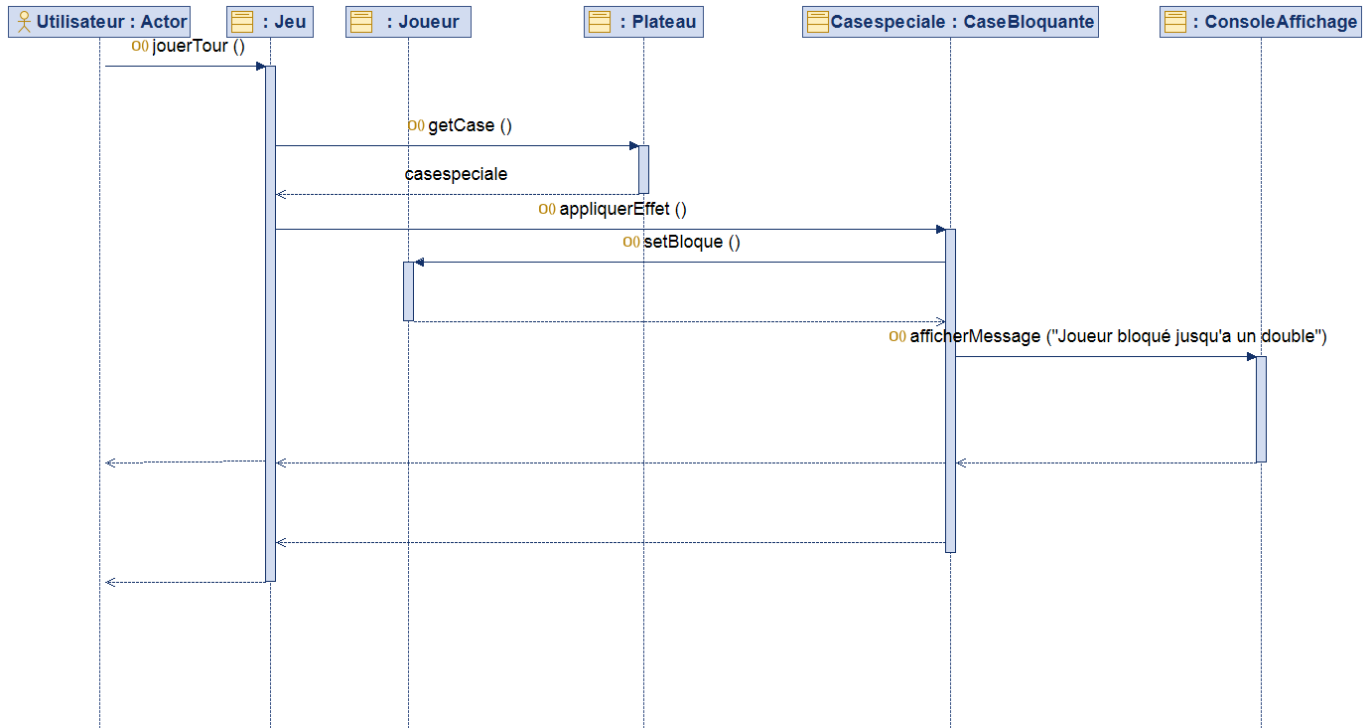
Diagrammes de séquence - Case spéciale : Bloquante

Ce diagramme représente le comportement d'une CaseBloquante.

Lorsque le joueur arrive sur cette case, la méthode `appliquerEffet(joueur, pion)` est appelée.

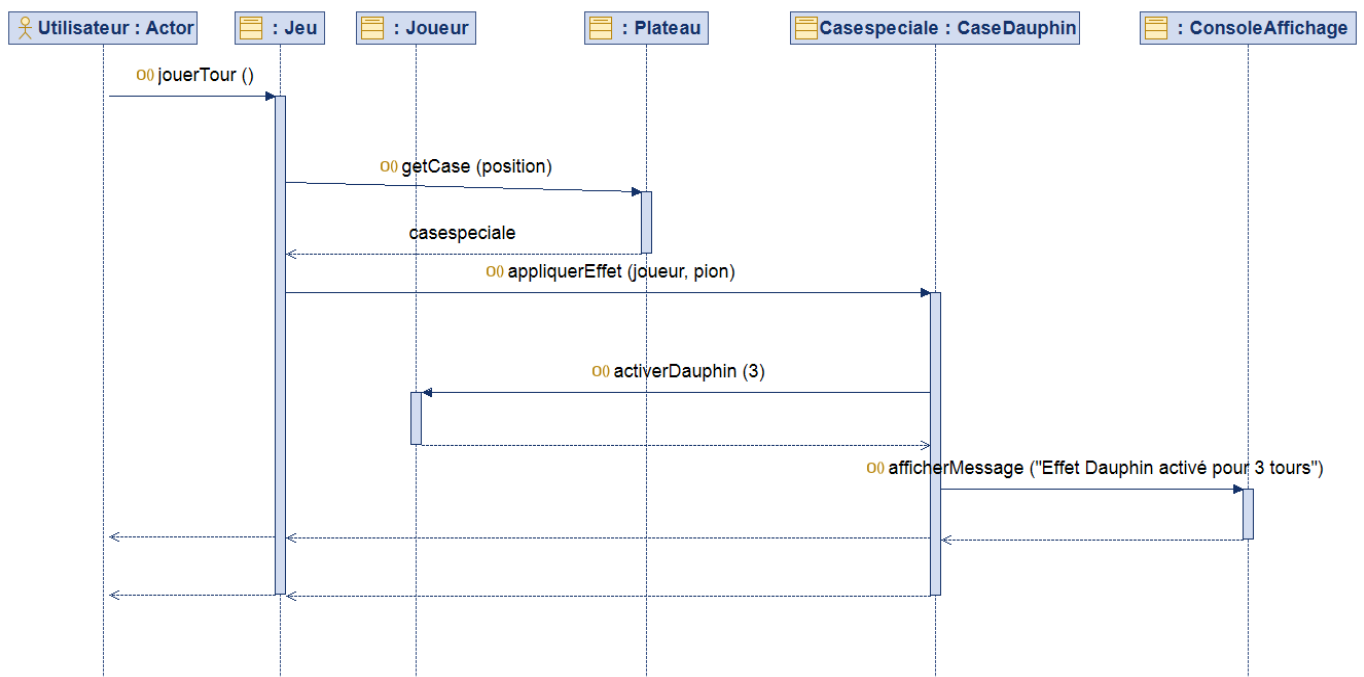
Cette méthode appelle `setBloque(true)` afin de mettre le joueur dans un état bloqué.

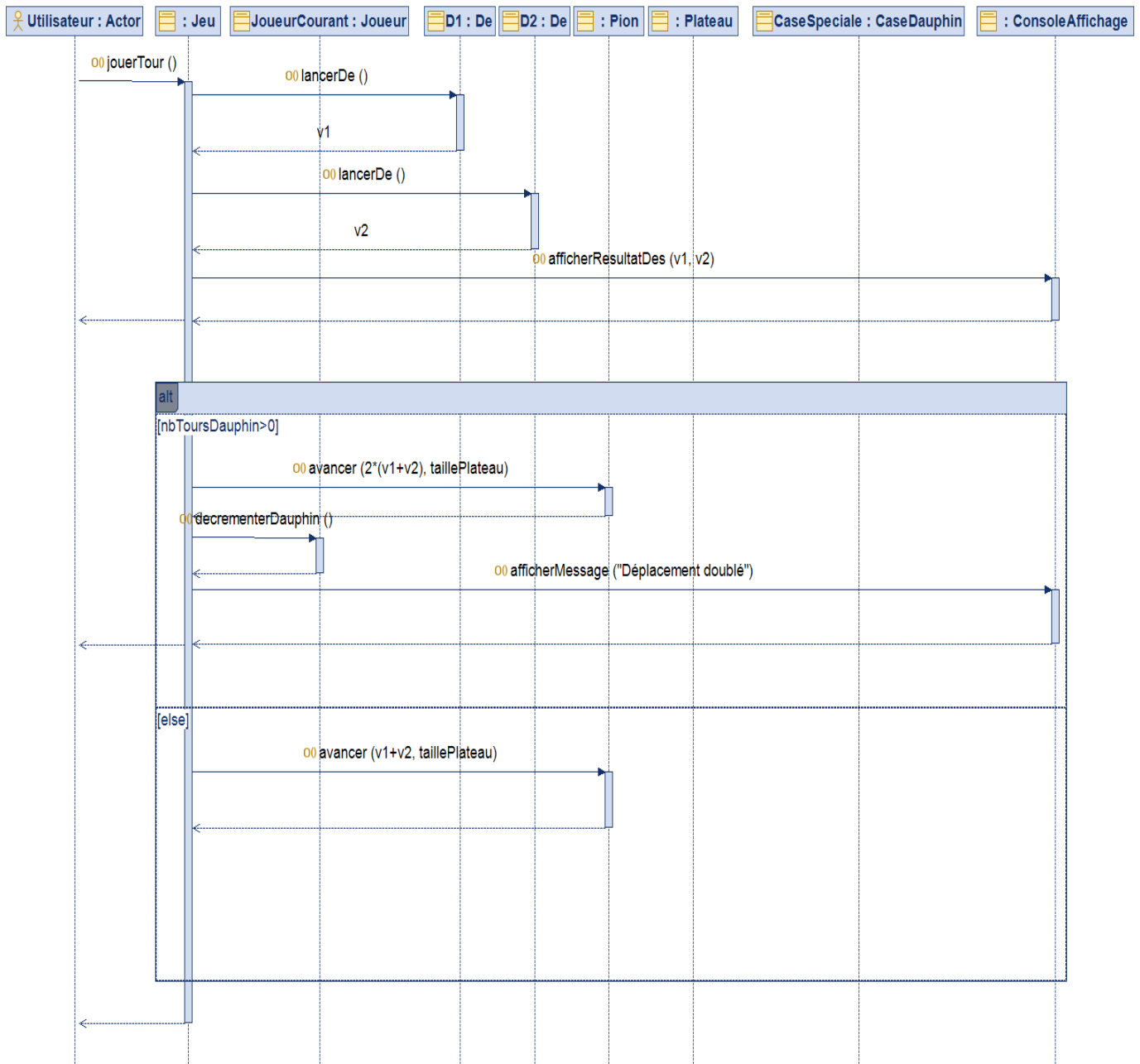
Un message est ensuite affiché pour indiquer que le joueur restera bloqué jusqu'à l'obtention d'un double.



Diagrammes de séquence - Case spéciale : Dauphin

Ce diagramme représente le comportement lorsqu'un joueur arrive sur une CaseDauphin. Le jeu récupère la case via `getCase(position)` puis appelle `appliquerEffet(joueur, pion)`. La méthode `activerDauphin(3)` est appelée, ce qui active l'effet pour 3 tours. Tant que `nbToursDauphin > 0`, le déplacement du joueur est doublé. À chaque tour, la méthode `decrementerDauphin()` est appelée pour réduire le compteur.





Le fragment alt [nbToursDauphin > 0] permet de modéliser les deux comportements :

- Déplacement doublé
- Déplacement normal

Conclusion

Ce projet a permis de modéliser un jeu complet en utilisant UML.

Les diagrammes de classes et de séquence permettent de représenter clairement :

- La structure du système,
- Le comportement dynamique du jeu.